

TMM4128:

Machine Learning for Engineers

Andrei Lobov

e-mail: andrei.lobov@ntnu.no

phone: +47 45 913 455

room: 213, Verkstedteknisk

web: <http://www.lobov.biz/academia>

Slides:> <http://lobov.biz/academia/mle/250124>

Outline : Training models

- Linear Regression
 - Model of life satisfaction
 - The Normal Equation
 - Computational Complexity
- Gradient Descent
 - Batch Gradient Descent
 - Stochastic Gradient Descent
 - Mini-batch Gradient Descent
- Polynomial Regression
- Learning Curves
- Homework

—

Source: Aurelien Geron, “Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow : concepts, tools, and techniques to build intelligent systems”, 2019 ([Available online](#)) / **Chapters 1 and 4**

Linear Regression

Ch1: Steps 1-28; Ch4: Steps 1-12

Does money make people happier?

Table 1-1. Does money make people happier?

Country	GDP per capita (USD)	Life satisfaction
Hungary	12,240	4.9
Korea	27,195	5.8
France	37,675	6.5
Australia	50,962	7.3
United States	55,805	7.2

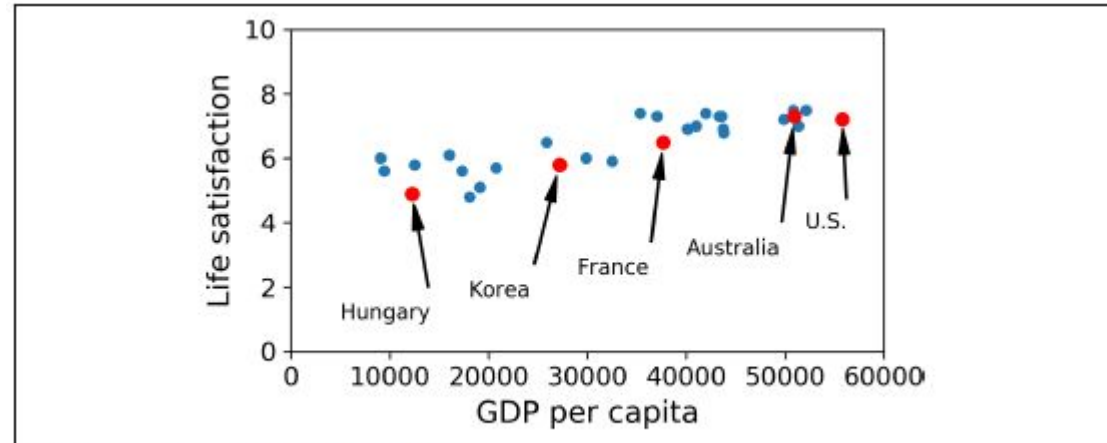


Figure 1-17. Do you see a trend here?

A simple linear model - Model selection: Linear model

Equation 1-1. A simple linear model

$$\text{life_satisfaction} = \theta_0 + \theta_1 \times \text{GDP_per_capita}$$

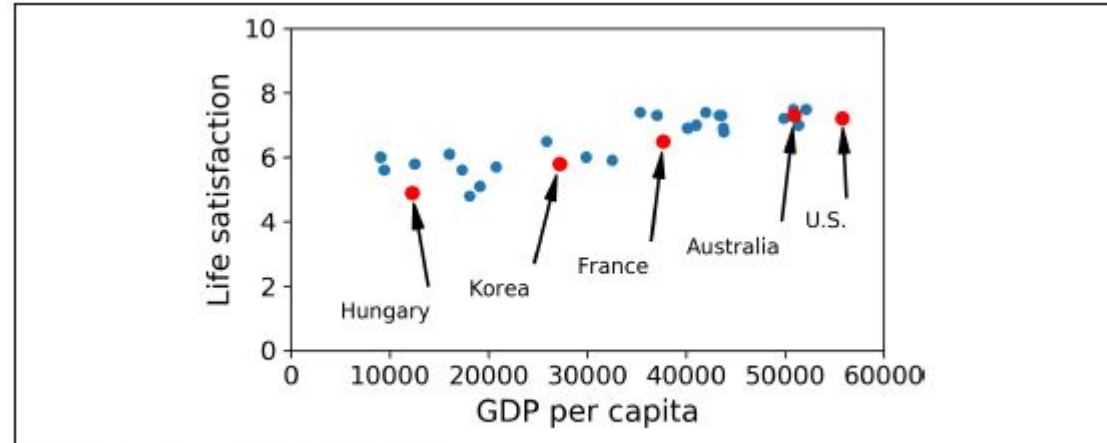


Figure 1-17. Do you see a trend here?

A few possible linear models

Two parameters to tweak - θ_0, θ_1 . [θ - pronounced as *theta*]

Equation 1-1. A simple linear model

$$\text{life_satisfaction} = \theta_0 + \theta_1 \times \text{GDP_per_capita}$$

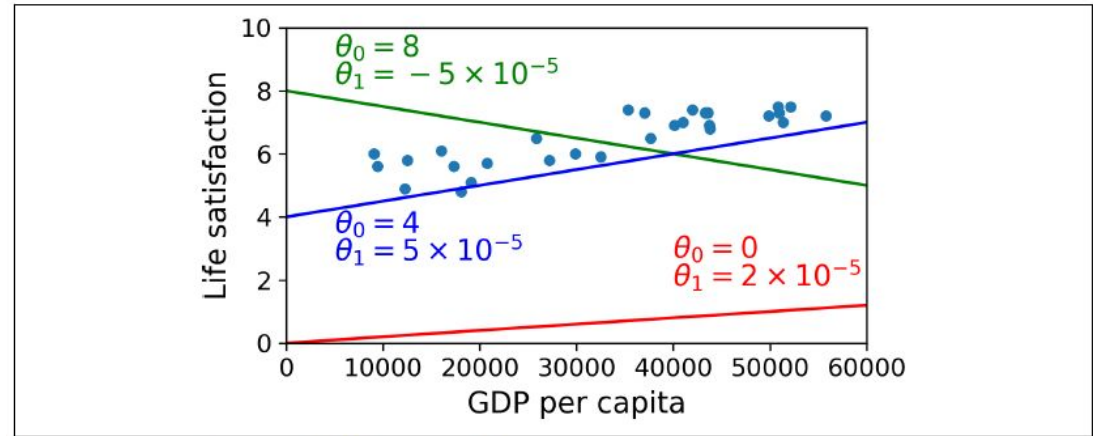


Figure 1-18. A few possible linear models

One can either define a **utility function** (or **fitness function**) that measures how good the model is, or you can define a **cost function** that measures how bad it is. For linear regression problems, typically - a cost function that measures the distance between the linear model's predictions and the training examples; objective - to minimize this distance.

Training

Linear Regression: feed it your training examples and it finds the parameters that make the linear model fit best to the data.

Result $\theta_0 = 4.85$ and $\theta_1 = 4.91 \times 10^{-5}$.

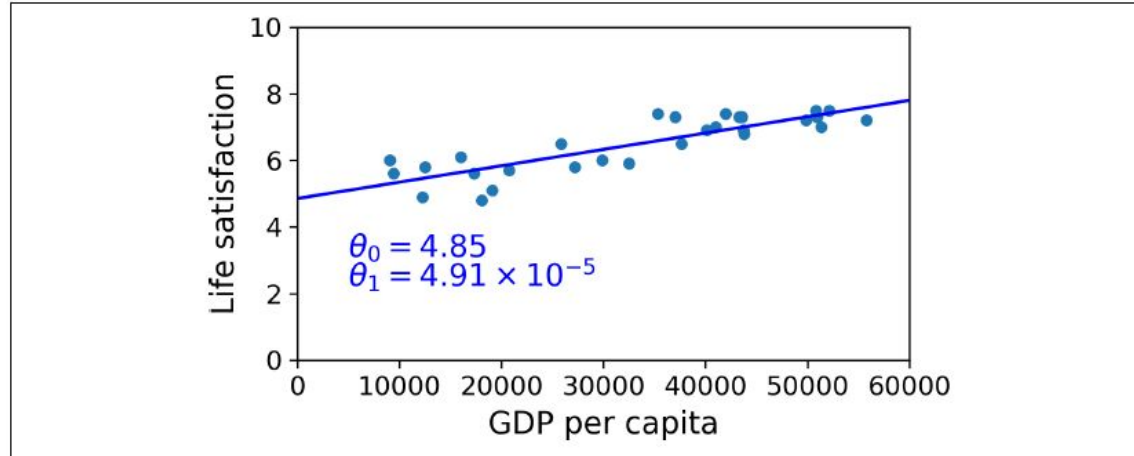
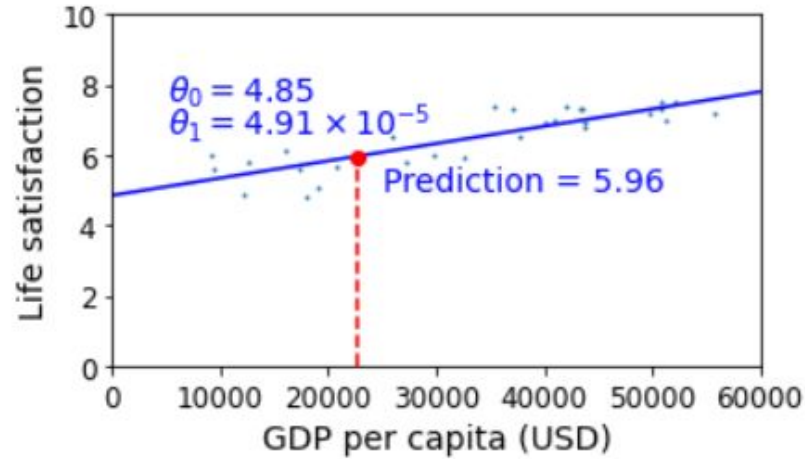


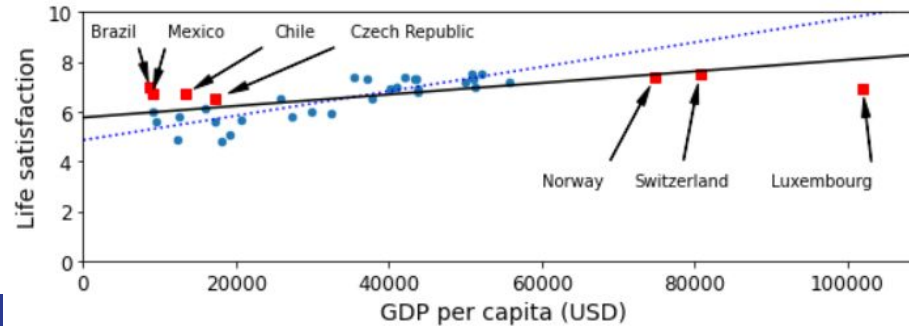
Figure 1-19. The linear model that fits the training data best

Making predictions

Cyprus:>



yes, but:>



Linear Regression

Equation 1-1. A simple linear model

$$\text{life_satisfaction} = \theta_0 + \theta_1 \times \text{GDP_per_capita}$$

Equation 4-1. Linear Regression model prediction

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

- $\hat{y}$ is the predicted value.
- n is the number of features.
- x_i is the i^{th} feature value.
- θ_j is the j^{th} model parameter (including the *bias term* θ_0 and the feature weights $\theta_1, \theta_2, \dots, \theta_n$).

Linear Regression: Vectorized form

Equation 4-2. Linear Regression model prediction (vectorized form)

$$\hat{y} = h_{\theta}(\mathbf{x}) = \boldsymbol{\theta} \cdot \mathbf{x}$$

- $\boldsymbol{\theta}$ is the model's parameter vector, containing the bias term θ_0 and the feature weights θ_1 to θ_n .
 - $\mathbf{x}$ is the instance's feature vector, containing x_0 to x_n , with x_0 always equal to 1.
 - $\boldsymbol{\theta} \cdot \mathbf{x}$ is the dot product of the vectors $\boldsymbol{\theta}$ and $\mathbf{x}$, which is of course equal to $\theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$.
 - h_{θ} is the hypothesis function, using the model parameters θ .
- $\hat{y} = \boldsymbol{\theta}^T \mathbf{x}$, - often in use (Transpose form) as vectors often represented as *column vectors*.

Linear Regression: Performance measure

$$\text{RMSE}(\mathbf{X}, h) = \sqrt{\frac{1}{m} \sum_{i=1}^m \left(h(\mathbf{x}^{(i)}) - y^{(i)} \right)^2}$$

Value of $\boldsymbol{\theta}$ to minimize the RMSE?

RMSE or MSE?: Easier with MSE in practice than with RMSE, and leads to the same result as $[\min f^2(v) \sim \min(\text{sqrt}(f^2(v)))]$

Equation 4-3. MSE cost function for a Linear Regression model

$$\text{MSE}(\mathbf{X}, h_{\boldsymbol{\theta}}) = \frac{1}{m} \sum_{i=1}^m \left(\boldsymbol{\theta}^T \mathbf{x}^{(i)} - y^{(i)} \right)^2$$

The Normal Equation: gives the result directly

Derivation.

Equation 4-4. Normal Equation

$$\hat{\boldsymbol{\theta}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

- $\hat{\boldsymbol{\theta}}$ is the value of $\boldsymbol{\theta}$ that minimizes the cost function.
- $\mathbf{y}$ is the vector of target values containing $y^{(1)}$ to $y^{(m)}$.

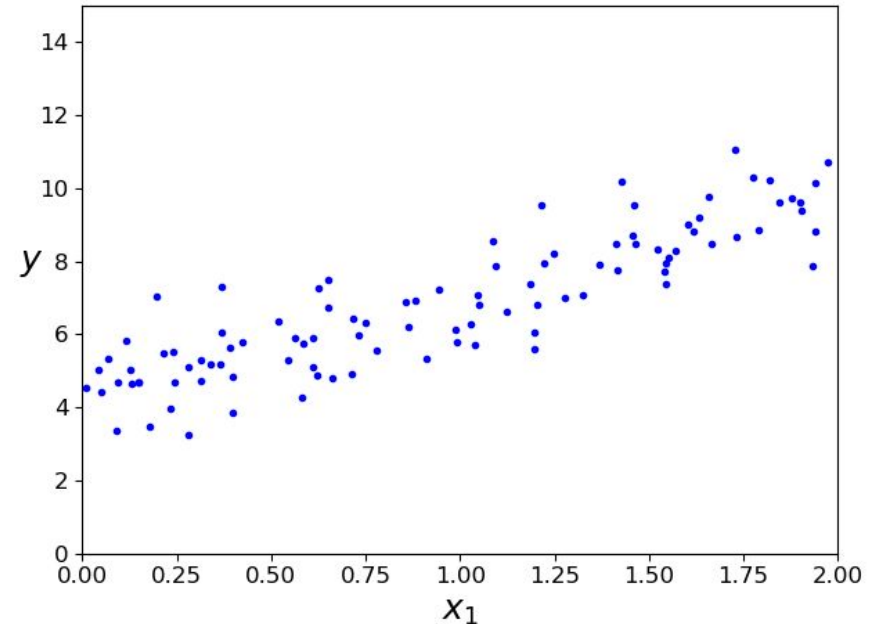
Linear Regression

1. Getting the data
2. Computing $\hat{\theta}$
3. Making predictions

```
import numpy as np
```

```
X = 2 * np.random.rand(100, 1)
```

```
y = 4 + 3 * X + np.random.randn(100, 1)
```



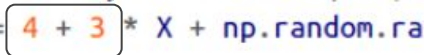
Linear Regression

1. Getting the data
2. **Computing $\hat{\theta}$**
3. Making predictions

```
X_b = np.c_[np.ones((100, 1)), X] # add x0 = 1 to each instance  
theta_best = np.linalg.inv(X_b.T.dot(X_b)).dot(X_b.T).dot(y)
```

```
>>> theta_best  
array([[4.21509616],  
       [2.77011339]])
```

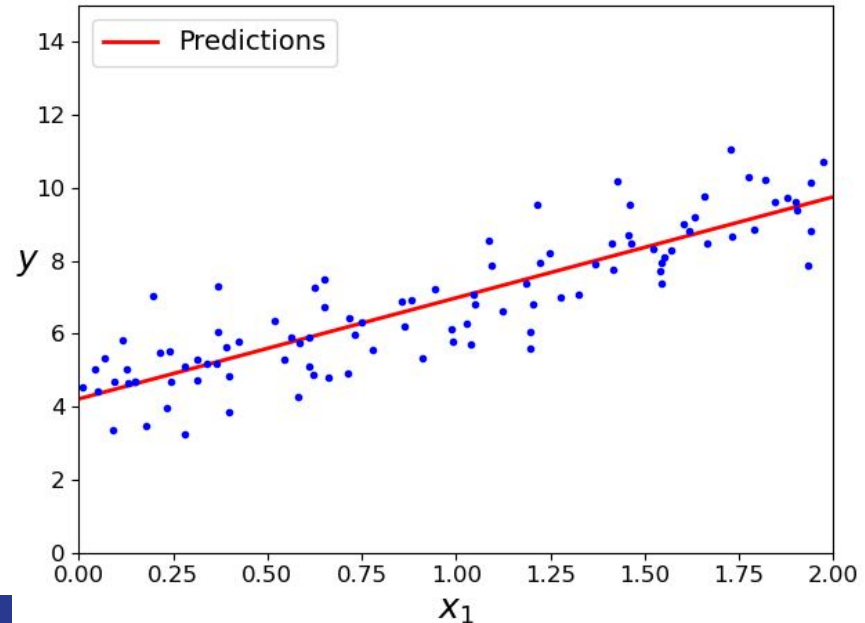
```
y = 4 + 3 * X + np.random.randn(100, 1)
```



Linear Regression

1. Getting the data
2. Computing $\hat{\theta}$
3. Making predictions
 - a. $y = 4 + 3 * 0 = 4$
 - b. $y = 4 + 3 * 2 = 10$

```
>>> X_new = np.array([[0], [2]])
>>> X_new_b = np.c_[np.ones((2, 1)), X_new] # add x0 = 1 to each instance
>>> y_predict = X_new_b.dot(theta_best)
>>> y_predict
array([[4.21509616],
       [9.75532293]])
```



Using LinearRegression of Scikit-Learn

intercept_ : bias term

coef_ : feature weights

```
from sklearn.linear_model import LinearRegression  
  
lin_reg = LinearRegression()  
lin_reg.fit(X, y)  
lin_reg.intercept_, lin_reg.coef_  
  
(array([4.21509616]), array([[2.77011339]]))
```

```
>>> theta_best  
array([[4.21509616],  
       [2.77011339]])
```

```
lin_reg.predict(X_new)  
  
array([[4.21509616],  
       [9.75532293]])
```

```
>>> y_predict  
array([[4.21509616],  
       [9.75532293]])
```


Singular Value Decomposition (SVD) instead of the Normal Equation

More efficient than the Normal Equation, no singularities (i.e. if $\mathbf{X}^T\mathbf{X}$ is not invertible).

Computes $\hat{\boldsymbol{\theta}} = \mathbf{X}^+\mathbf{y}$, where $\mathbf{X}^+$ is pseudoinverse of $\mathbf{X}$.

```
>>> theta_best_svd, residuals, rank, s = np.linalg.lstsq(X_b, y, rcond=1e-6)
>>> theta_best_svd
array([[4.21509616],
       [2.77011339]])
```

```
>>> np.linalg.pinv(X_b).dot(y)
array([[4.21509616],
       [2.77011339]])
```

Computational Complexity

Computing the inverse of $\mathbf{X}^T\mathbf{X}$: $(n+1) \times (n+1)$ matrix, where n is the number of features.

Computational complexity of inverting such a matrix : about $O(n^{2.4})$ to $O(n^3)$.

$2 * n \sim 2^{2.4} .. 2^3 * \text{computation time. (In training)}$

$n \sim 100'000$: the Normal Equation or SVD-based approaches get very slow.
However, they are linear w.r.t. number of instances, thus handle efficiently large data sets if these fit the memory.

Also, once trained, it is fast to make predictions:

$2 * \text{instances} \sim 2 * \text{prediction (computation) time}$

Gradient Descent

Steps 13-26

Gradient Descent

Start by filling θ with random values (random initialization). Take small step at the time to decrease the cost function (e.g. the MSE).

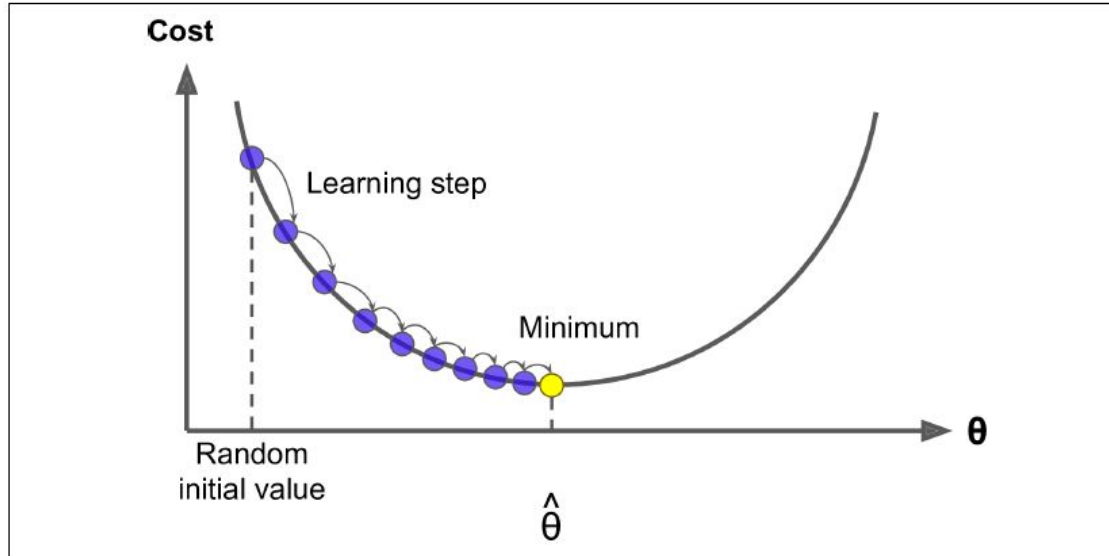


Figure 4-3. Gradient Descent

Gradient Descent : Size of the stem

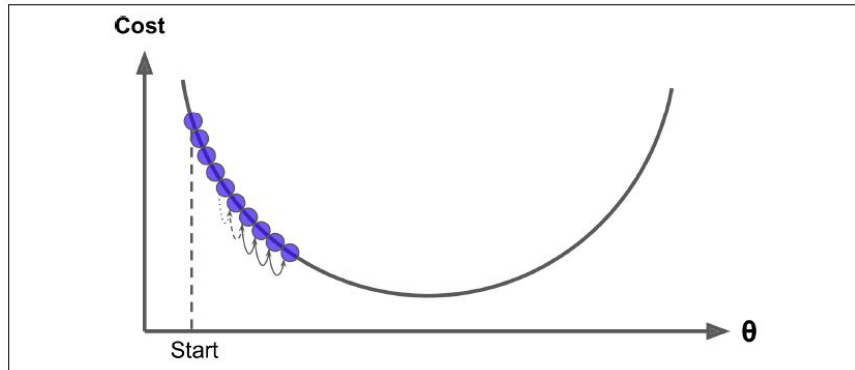


Figure 4-4. Learning rate too small

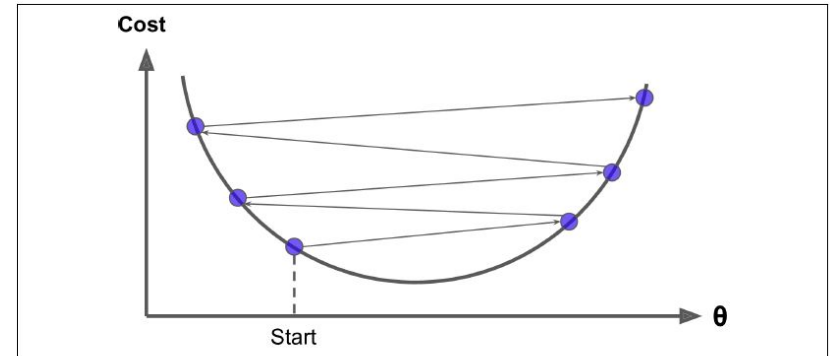


Figure 4-5. Learning rate too large

Gradient Descent : Pitfalls

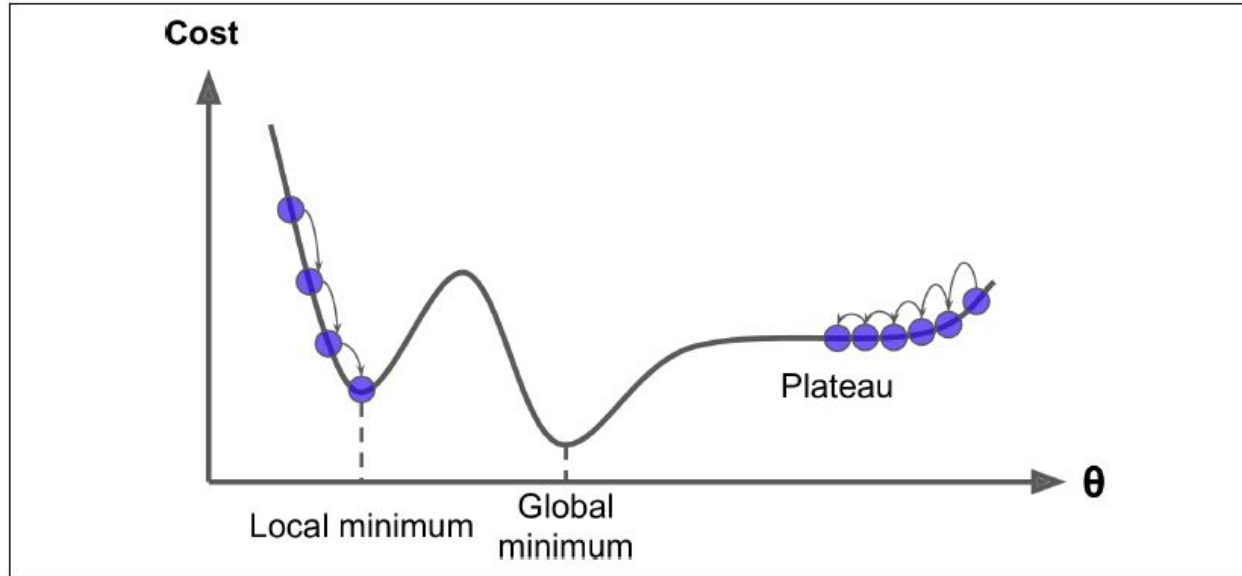


Figure 4-6. Gradient Descent pitfalls

Gradient Descent is guaranteed to approach arbitrarily close the global minimum (if you wait long enough and if the learning rate is not too high).

Gradient Descent : Feature scaling

StandardScaler

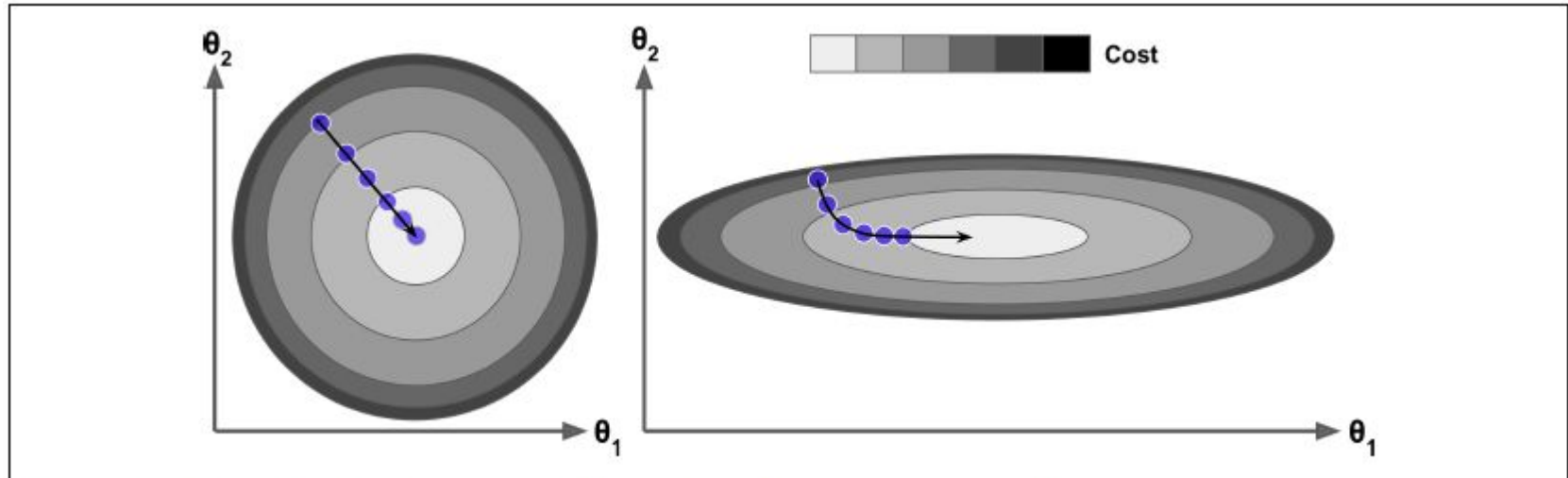


Figure 4-7. Gradient Descent with and without feature scaling

Gradient Descent : Gradient of the cost function

Equation 4-5. Partial derivatives of the cost function

$$\frac{\partial}{\partial \theta_j} \text{MSE}(\boldsymbol{\theta}) = \frac{2}{m} \sum_{i=1}^m (\boldsymbol{\theta}^T \mathbf{x}^{(i)} - y^{(i)}) x_j^{(i)}$$

Equation 4-6. Gradient vector of the cost function

$$\nabla_{\boldsymbol{\theta}} \text{MSE}(\boldsymbol{\theta}) = \begin{pmatrix} \frac{\partial}{\partial \theta_0} \text{MSE}(\boldsymbol{\theta}) \\ \frac{\partial}{\partial \theta_1} \text{MSE}(\boldsymbol{\theta}) \\ \vdots \\ \frac{\partial}{\partial \theta_n} \text{MSE}(\boldsymbol{\theta}) \end{pmatrix} = \frac{2}{m} \mathbf{X}^T (\mathbf{X}\boldsymbol{\theta} - \mathbf{y})$$

Equation 4-7. Gradient Descent step

$$\boldsymbol{\theta}^{(\text{next step})} = \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} \text{MSE}(\boldsymbol{\theta})$$

Gradient Descent : Implementation

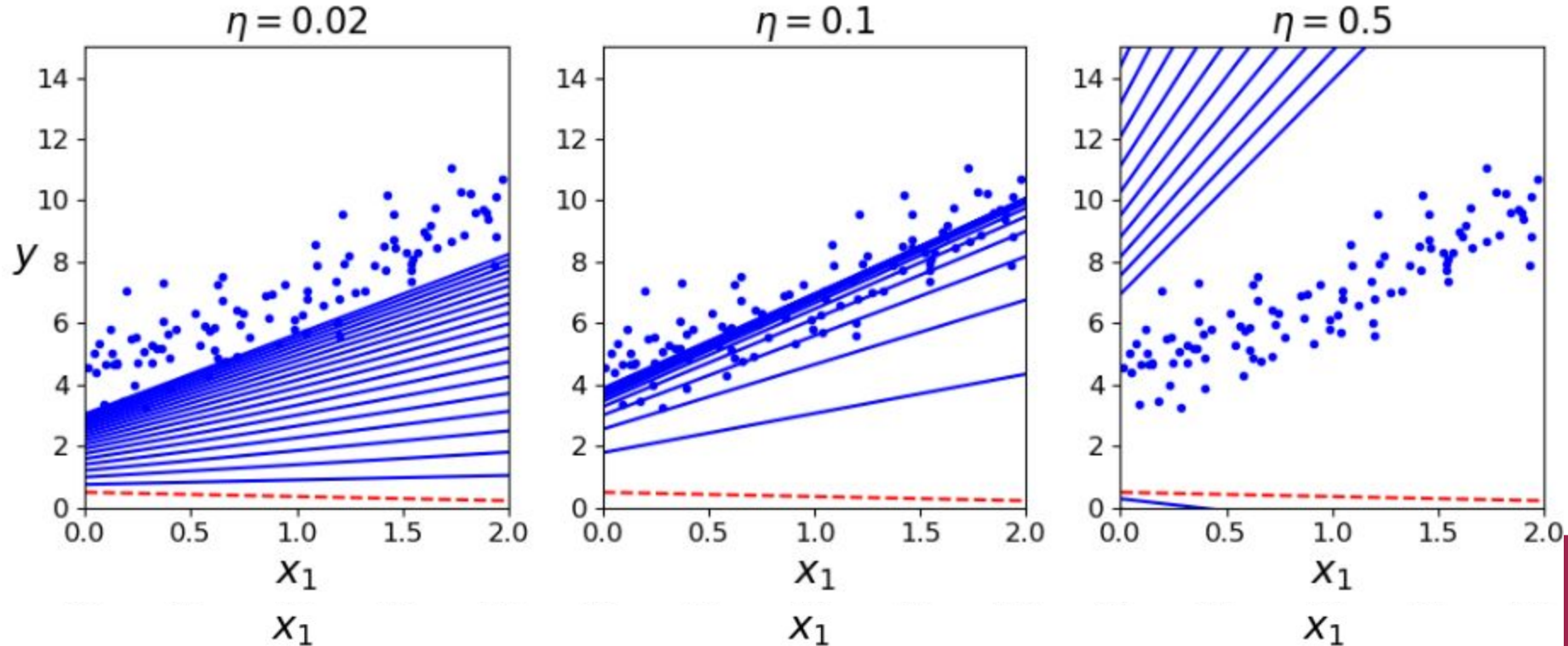
```
eta = 0.1 # learning rate
n_iterations = 1000
m = 100

theta = np.random.randn(2,1) # random initialization

for iteration in range(n_iterations):
    gradients = 2/m * X_b.T.dot(X_b.dot(theta) - y)
    theta = theta - eta * gradients

>>> theta
array([[4.21509616],
       [2.77011339]])
```

Gradient Descent : Different learning rates



Stochastic Gradient Descent

Having better chances of finding global minimum (Fig. 4-6) than Batch Gradient Descent. Gradient is calculated based on a single randomly-chosen instance.

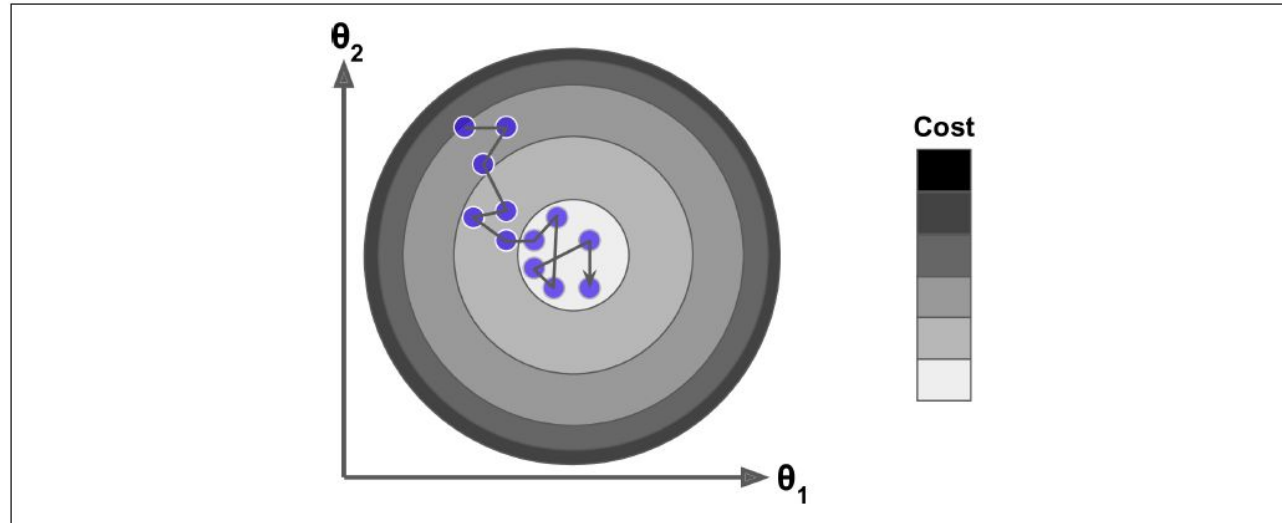


Figure 4-9. Stochastic Gradient Descent

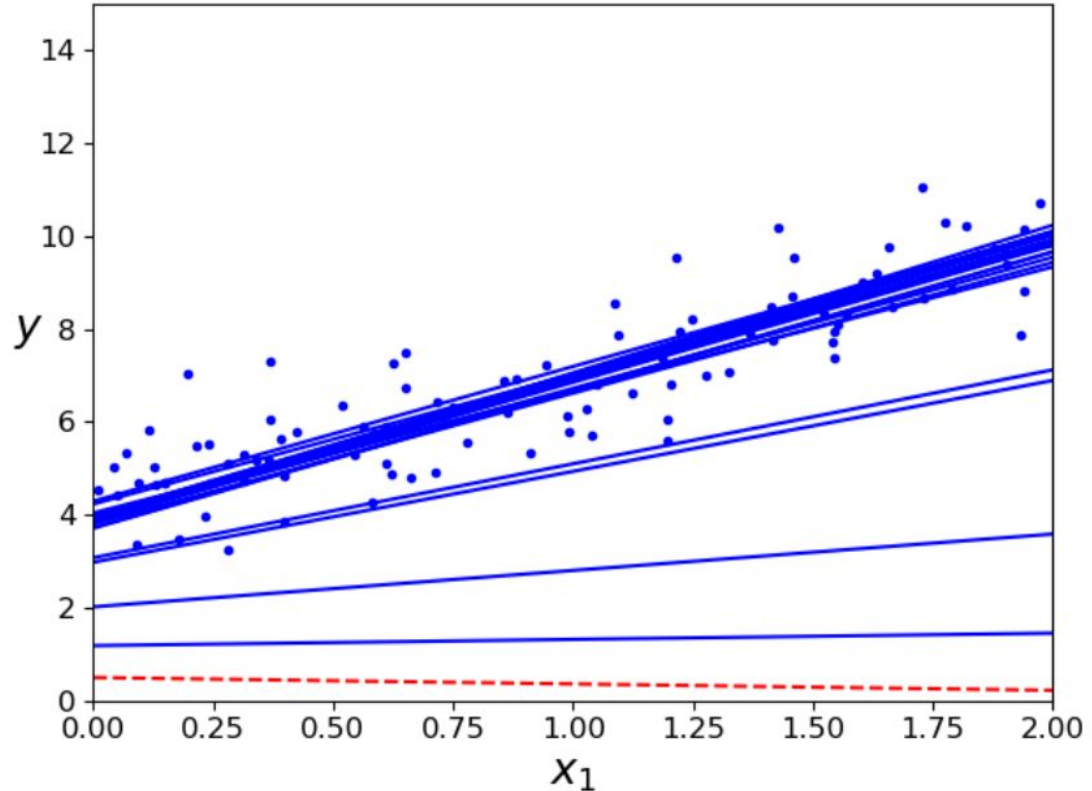
Stochastic Gradient Descent : Implementation (1)

```
n_epochs = 50
t0, t1 = 5, 50 # learning schedule hyperparameters

def learning_schedule(t):
    return t0 / (t + t1)

theta = np.random.randn(2,1) # random initialization

for epoch in range(n_epochs):
    for i in range(m):
        random_index = np.random.randint(m)
        xi = X_b[random_index:random_index+1]
        yi = y[random_index:random_index+1]
        gradients = 2 * xi.T.dot(xi.dot(theta) - yi)
        eta = learning_schedule(epoch * m + i)
        theta = theta - eta * gradients
```



Stochastic Gradient Descent : Implementation (2)

SGDRegressor

intercept_ : **bias term**

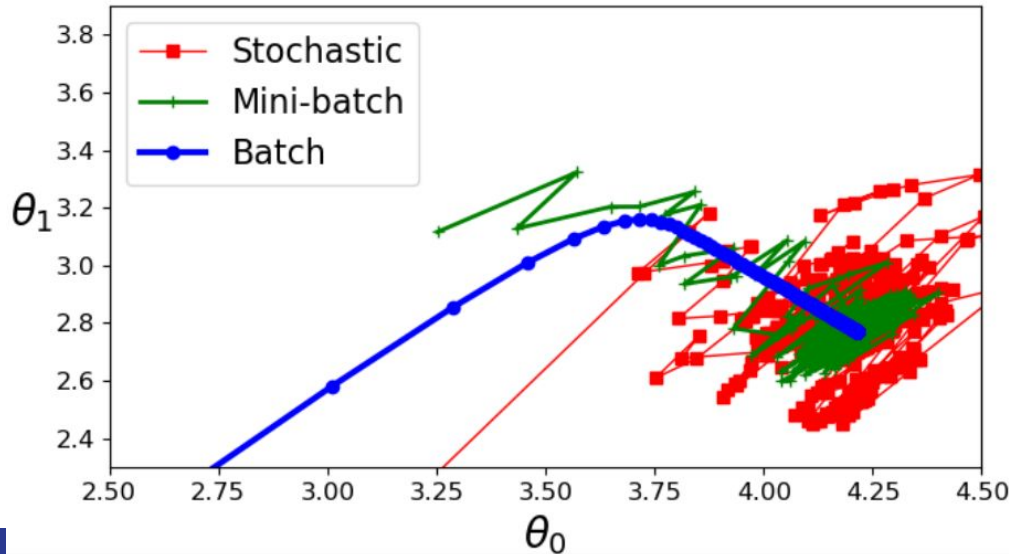
coef_ : **feature weights**

```
from sklearn.linear_model import SGDRegressor  
sgd_reg = SGDRegressor(max_iter=1000, tol=1e-3, penalty=None, eta0=0.1)  
sgd_reg.fit(X, y.ravel())
```

```
>>> sgd_reg.intercept_, sgd_reg.coef_  
(array([4.24365286]), array([2.8250878]))
```

Mini-batch Gradient Descent : Steps 23-26

At each step, instead of computing the gradients based on the full training set (as in Batch GD) or based on just one instance (as in Stochastic GD), Minibatch GD computes the gradients on small random sets of instances called *mini-batches*.



Comparison of algorithms for Linear Regression

m is the number of training instances and n is the number of features

Table 4-1. Comparison of algorithms for Linear Regression

Algorithm	Large m	Out-of-core support	Large n	Hyperparams	Scaling required	Scikit-Learn
Normal Equation	Fast	No	Slow	0	No	n/a
SVD	Fast	No	Slow	0	No	LinearRegression
Batch GD	Slow	No	Fast	2	Yes	SGDRegressor
Stochastic GD	Fast	Yes	Fast	≥ 2	Yes	SGDRegressor
Mini-batch GD	Fast	Yes	Fast	≥ 2	Yes	SGDRegressor

Definition: Out-of-core learning

Training systems on huge datasets that cannot fit in one machine's main memory (this is called out-of-core learning). Out-of-core is usually done *offline* - so, it is rather an "*incremental learning*".

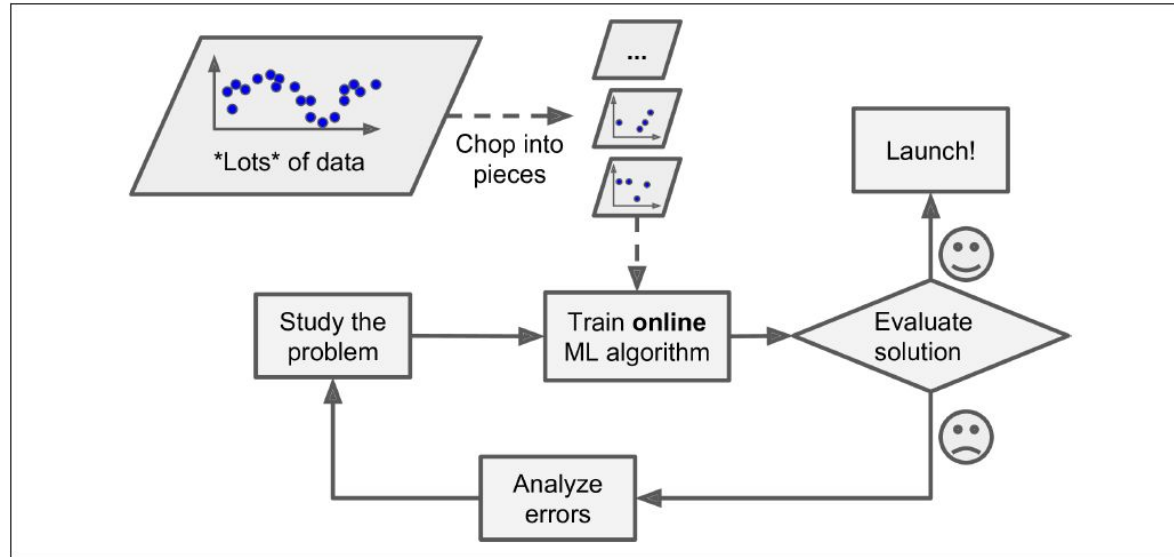


Figure 1-14. Using online learning to handle huge datasets

Polynomial Regression

Steps 27-34

Polynomial Regression

More complex data than a line?

Quadratic equation + some noise:

```
m = 100
X = 6 * np.random.rand(m, 1) - 3
y = 0.5 * X**2 + X + 2 + np.random.randn(m, 1)
```

$$y = 0.5x_1^2 + 1.0x_1 + 2.0 + \text{Gaussian noise.}$$

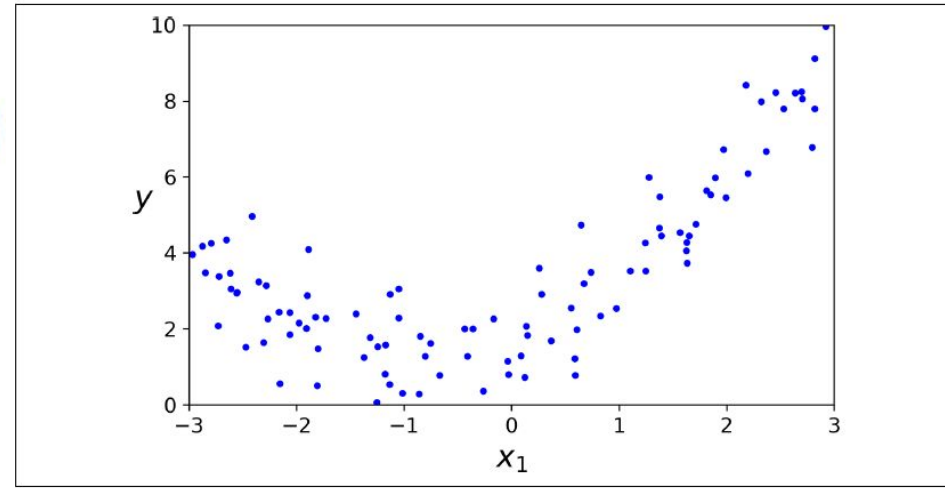


Figure 4-12. Generated nonlinear and noisy dataset

Polynomial Regression

Using PolynomialFeatures to transform training data - adding a new feature (in this example just one corresponding for the 2nd degree polynomial.)

```
>>> from sklearn.preprocessing import PolynomialFeatures
>>> poly_features = PolynomialFeatures(degree=2, include_bias=False)
>>> X_poly = poly_features.fit_transform(X)
>>> X[0]
array([-0.75275929])
>>> X_poly[0]
array([-0.75275929, 0.56664654])
```

Then, fitting the LinearRegression with the extended training data:

```
>>> lin_reg = LinearRegression()
>>> lin_reg.fit(X_poly, y)
>>> lin_reg.intercept_, lin_reg.coef_
(array([1.78134581]), array([[0.93366893, 0.56456263]]))
```

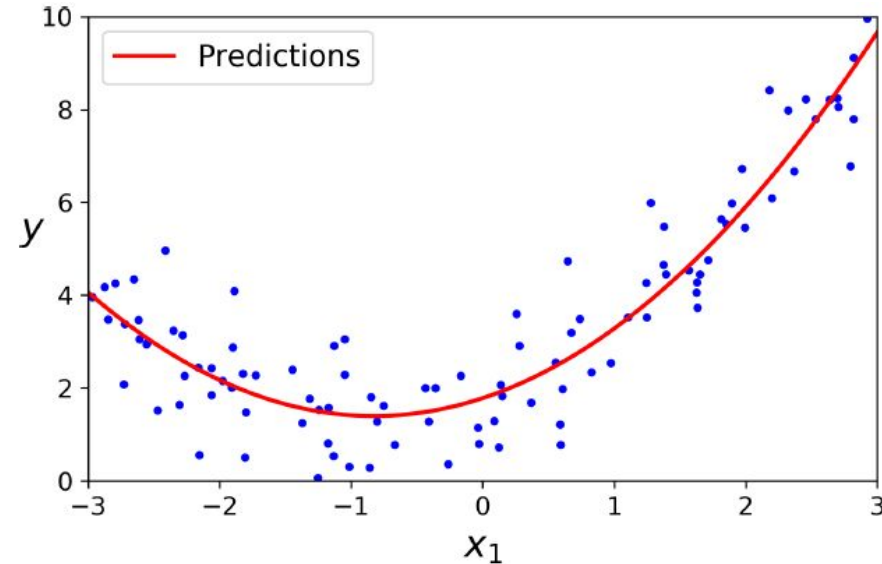
Polynomial Regression : Making predictions

```
X_new=np.linspace(-3, 3, 100).reshape(100, 1)
X_new_poly = poly_features.transform(X_new)
y_new = lin_reg.predict(X_new_poly)
plt.plot(X, y, "b.")
plt.plot(X_new, y_new, "r-", linewidth=2, label="Predictions")
plt.xlabel("$x_1$", fontsize=18)
plt.ylabel("$y$", rotation=0, fontsize=18)
plt.legend(loc="upper left", fontsize=14)
plt.axis([-3, 3, 0, 10])
save_fig("quadratic_predictions_plot")
plt.show()
```

```
>>> lin_reg = LinearRegression()
>>> lin_reg.fit(X_poly, y)
>>> lin_reg.intercept_, lin_reg.coef_
(array([1.78134581]), array([[0.93366893, 0.56456263]]))
```

$$\hat{y} = 0.56x_1^2 + 0.93x_1 + 1.78$$

$$y = 0.5x_1^2 + 1.0x_1 + 2.0 + \text{Gaussian noise.}$$



Learning Curves

Steps 34-37

High-degree Polynomial Regression

2nd degree fits the data best. Makes sense, but in general this is not known in advance.

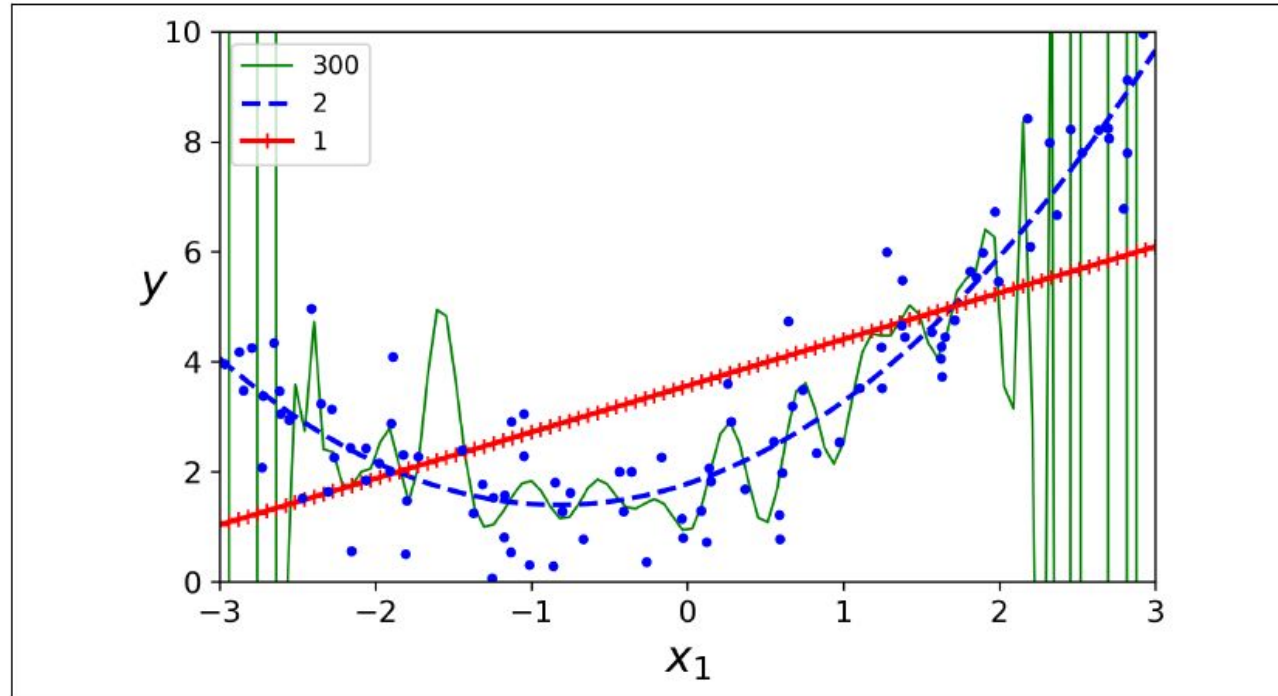


Figure 4-14. High-degree Polynomial Regression

Model overfitting or underfitting the data?

If a model performs well on the training data but generalizes poorly according to the cross-validation metrics, then your model is **overfitting**.

If the model performs poorly on both, then it is **underfitting**.
(A way to tell when a model is too simple or too complex.)

If the model is **underfitting** the training data, adding more training examples will not help. A more complex model or better features are needed.

A way to improve an **overfitting** model is to feed it more training data until the validation error reaches the training error.

Learning curves (underfitting)

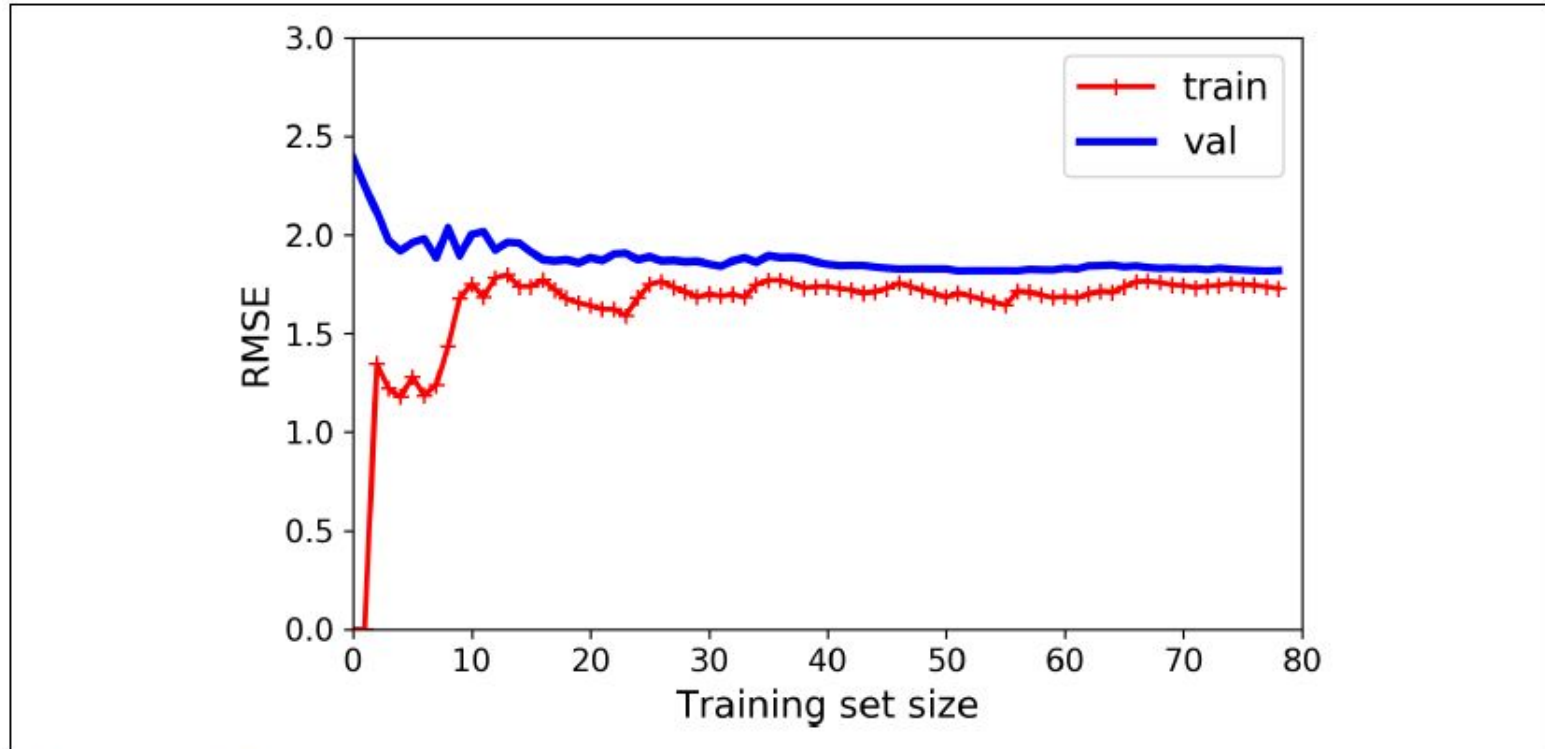


Figure 4-15. Learning curves

Learning curves (overfitting? more data?)

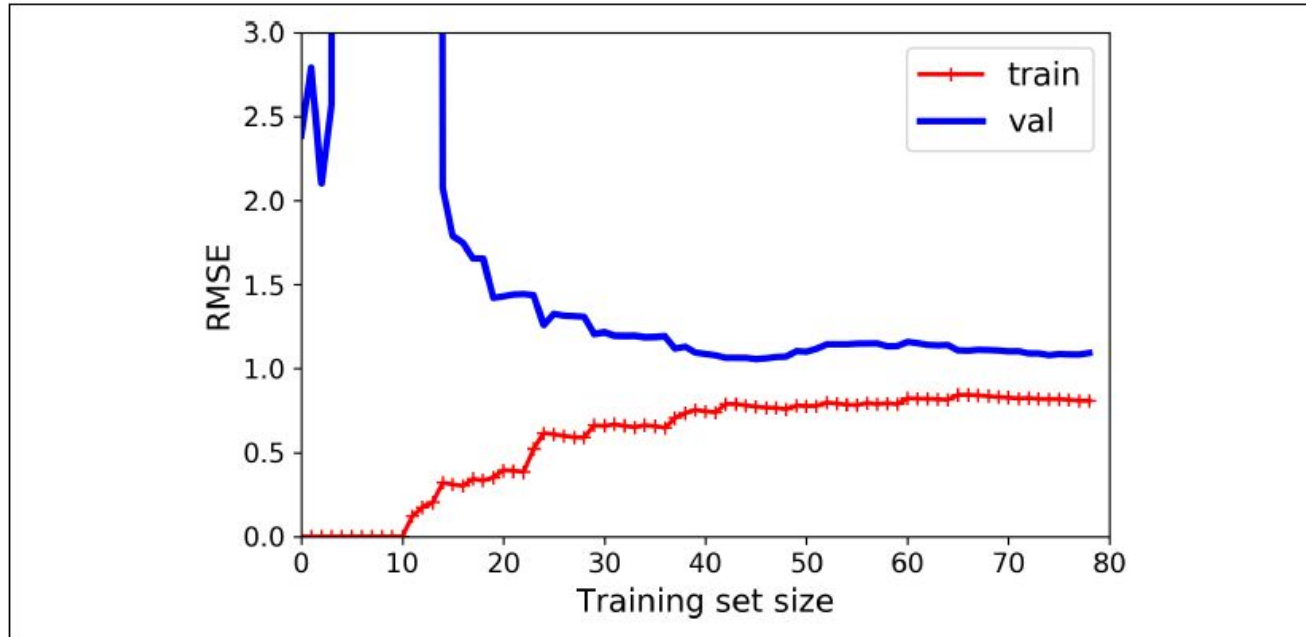


Figure 4-16. Learning curves for the polynomial model

Takeaways

- Principles of Linear and Polynomial Regression, (Stochastic) Gradient Descent models, their validation techniques.
- Use of learning curves.
- Use in and with Scikit-Learn.

Homework

Check Google Scholar for scientific publications on application of Linear or Polynomial Regression in product design, development. *You may need to be on NTNU network (i.e. either on campus or via the VPN) to have access to most of the articles.* Select and study one article.

For the article selected, check:

- a. for the “objective in business terms” - What and why they try to solve / improve?
- b. how is the linear and/or polynomial regression applied - how does it perform?
- c. what ML tools / “frameworks” or toolchains are in use?
- d. what are the features and labels?

Format of submission: **Max 1 A4** page PDF or DOCX answering the questions above.

DL: **Wed. 29.01., 12:00** to be submitted via Blackboard.